Universidad Nacional del Nordeste

Facultad de Ciencias Exactas y Naturales y Agrimensura

Lic. en Sistemas de Información

Ingeniería de Software 2

2026

Grupo Nº 43

Integrantes:

Apellido y Nombre	**Número de DNI**
Zini, Samuel Nehuen	44.196.064
Orban, Tobias Naim	46.385.637

Proyecto de Trabajo de Campo

Contenido:

1. Ciclo de Vida del Proyecto **3**
2. Planificación por Sprint (Scrum) **3**
2.1. Sprint 1: Registro y Gestión Básica de Pacientes 4
2.2. Sprint 2: Asignación y Gestión de Habitaciones 4
2.3. Sprint 3: Gestión de Egreso de Pacientes 4
2.4. Sprint 4: Implementación del Historial Clínico 5
2.5. Sprint 5: Integración y Validación del Sistema 5
3. Alcance de Funcionalidades Pretendido **6**
3.1. Funcionalidad 1: Gestión de Pacientes 6
3.2. Funcionalidad 2: Historial Clínico del Paciente 6
4. Plan de Riesgos **7**
4.1. Identificación de Riesgos 7
4.2. Análisis de Riesgos 7
4.3. Estrategias de Mitigación 8
4.4. Monitoreo de Riesgos 8
5. Evaluación de Arquitecturas, Selección y Justificación **9**
5.1. Arquitecturas Seleccionadas y Justificación 9
5.1.1. Arquitectura por Capas 9
5.1.2. Arquitectura Cliente-Servidor 10
6. Base de datos **11**
6.1. Modelo lógico: diagrama de entidad relación 11
6.2. Diccionario de datos 11
Tabla: persona 11
Tabla: provincia 11
Tabla: localidad 12
Tabla: domicilio 12
Tabla: obra_social 12
Tabla: antecedente_paciente 13
Tabla: paciente 13
Tabla: historial_medico 13
Tabla: habitacion 14
Tabla: internacion 14
Tabla: tipo_procedimiento 15
Tabla: rol 15
Tabla: usuario 15
Tabla: registro_clinico 15
Tabla: medicacion 16
Tabla: telefono 16
Tabla: suministro_medicacion 16

1. Ciclo de Vida del Proyecto

El ciclo de vida adoptado para el desarrollo de Clinicks es un modelo de desarrollo ágil basado en Scrum. La elección de esta metodología se justifica por las siguientes razones de la naturaleza del entorno hospitalario:

- **Iteraciones cortas con entregas continuas**: Se planificaron *sprints* de 1 semana, lo que permite entregar funcionalidades utilizables de forma temprana (por ejemplo, tener un módulo básico de admisión de pacientes funcionando desde el primer sprint).
- **Adaptabilidad a cambios**: Al estar enfocado en distintos usuarios (médicos, enfermeros, administrativos), permite reajustar funcionalidades y prioridades según el *feedback* o nuevas normativas hospitalarias.
- **Comunicación constante**: Fomenta la revisión periódica con los usuarios del hospital al final de cada *sprint*, alineando el sistema a las necesidades reales y minimizando malentendidos.
- **Priorización de valor y calidad**: Permite priorizar funcionalidades críticas (ej. registro e internaciones primero) garantizando la trazabilidad y la calidad de los datos, que son aspectos críticos en salud.

---

2. Planificación por Sprint (Scrum)

La planificación del proyecto se estructura en torno al desarrollo de dos funcionalidades fundamentales: la **Gestión de Pacientes** y el **Historial Clínico del Paciente**, consideradas como el eje central del sistema hospitalario.

Se adoptará la metodología Scrum, operando en *sprints* de **una semana** de duración. El enfoque primordial es la construcción incremental y progresiva del sistema, iniciando con la gestión básica de la información de pacientes para luego evolucionar hacia la integración del historial clínico.

2.1. Sprint 1: Registro y Gestión Básica de Pacientes

Objetivo: Facilitar el proceso de registro de nuevos pacientes y la administración de sus datos personales.

Historias de Usuario incluidas:

- Registrar paciente (GP-01)
- Editar información de paciente (GP-03)

Tareas:

- Diseño y estructuración del formulario de registro.
- Implementación de la validación de campos obligatorios.
- Desarrollo del *backend* para el procesamiento y alta del paciente.
- Establecimiento de la conectividad con el sistema de gestión de bases de datos.
- Desarrollo de la funcionalidad de búsqueda por Identificación (DNI) o nombre.
- Implementación de la lógica para la edición y actualización de datos.

2.2. Sprint 2: Asignación y Gestión de Habitaciones

Objetivo:

Administrar eficientemente la asignación de unidades de internación a los pacientes que requieran hospitalización.

Historias de Usuario incluidas:

- Asignar habitación (GP-02)

Tareas:

- Desarrollo del listado de habitaciones con indicación de su disponibilidad.
- Implementación de la validación de la disponibilidad en tiempo real.
- Asignación formal de la habitación al paciente.
- Actualización del estado de la habitación en la base de datos.
- Desarrollo del manejo de excepciones y errores (ej. falta de disponibilidad).

2.3. Sprint 3: Gestión de Egreso de Pacientes

Objetivo:

Documentar el proceso de alta de pacientes y actualizar su estado dentro del sistema.

Historias de Usuario incluidas:

- Registrar egreso de paciente (GP-04)

Tareas:

- Registro de la fecha y hora precisas del egreso.
- Liberación automática del estado de ocupación de la habitación.
- Actualización del estado del paciente (ej. 'Alta', 'Egresado').
- Aseguramiento de la persistencia y accesibilidad del historial clínico post-egreso.

2.4. Sprint 4: Implementación del Historial Clínico

Objetivo:

Desarrollar la funcionalidad integral del historial clínico, permitiendo la centralización y visualización cronológica de todos los eventos médicos relevantes del paciente.

Historias de Usuario incluidas:

- Consultar historial clínico del paciente (PD-02)
- Registrar eventos clínicos (consultas, procedimientos, medicación) (PD-01)

Tareas:

- Diseño de la arquitectura y estructura de datos del historial clínico.
- Implementación del módulo de registro de nuevos eventos clínicos.
- Asociación inequívoca de cada evento con el paciente correspondiente.
- Implementación del ordenamiento cronológico de la información.
- Desarrollo de herramientas de filtrado por criterios (fecha, tipología del evento).
- Implementación de mecanismos de control de acceso basados en roles.

2.5. Sprint 5: Integración y Validación del Sistema

Objetivo:

Verificar la correcta integración y operatividad de las funcionalidades de gestión de pacientes y historial clínico.

Historias de Usuario incluidas:

- Visualizar línea de tiempo del historial (PD-03)
- Realizar pruebas de rendimiento (TE-01)

Tareas:

- Ejecución de pruebas integrales a nivel de sistema.
- Validación del flujo operativo completo: (registro → asignación → atención → historial → egreso).
- Identificación y corrección de defectos detectados.
- Optimización de la interfaz de usuario y la experiencia de usabilidad (*UX*).
- Realización de sesiones de validación con usuarios clave (*feedback*).

---

3. Alcance de Funcionalidades Pretendido

3.1. Funcionalidad 1: Gestión de Pacientes

El sistema permitirá la gestión integral de los pacientes, abarcando desde su registro inicial hasta la actualización de su estado dentro del hospital.

Alcance:

- Registrar nuevos pacientes con sus datos personales (nombre, apellido, DNI, dirección, teléfono).
- Editar y actualizar la información de pacientes existentes.
- Buscar y consultar pacientes mediante criterios como nombre, apellido o DNI.
- Registrar el ingreso del paciente al hospital.
- Registrar el egreso del paciente, incluyendo fecha y hora de alta.
- Asignar habitaciones a pacientes internados.
- Permitir la reasignación de habitaciones en caso de traslados internos.
- Gestionar el estado del paciente (internado, en tratamiento, dado de alta).

Límites:

- No se contemplan funcionalidades de facturación ni gestión administrativa externa.
- No incluye interacción directa con pacientes (uso exclusivo del personal del hospital).

3.2. Funcionalidad 2: Historial Clínico del Paciente

El sistema permitirá la visualización y gestión del historial clínico completo de cada paciente, centralizando toda la información médica en un único lugar, organizada de forma cronológica.

Alcance:

- Generar automáticamente un historial clínico por cada paciente registrado.
- Registrar y visualizar eventos clínicos, tales como:
    - Consultas médicas.
    - Diagnósticos.
    - Procedimientos realizados.
    - Medicación administrada.
    - Internaciones y cambios de habitación.
- Mostrar el historial clínico en orden cronológico (tipo línea de tiempo).
- Permitir la consulta del historial por parte de usuarios autorizados (médicos, enfermeros).
- Filtrar la información del historial por fecha, tipo de evento o profesional interviniente.
- Garantizar la trazabilidad de cada registro (quién lo realizó y cuándo).

Límites:

- No se incluyen integraciones con sistemas externos (laboratorios, obras sociales, etc.).
- No se contempla la edición o eliminación de registros clínicos históricos (solo consulta y carga).
- El acceso estará restringido según roles de usuario.

---

4. Plan de Riesgos

El plan de riesgos tiene como objetivo identificar, analizar y definir estrategias para mitigar los posibles problemas que puedan surgir durante el desarrollo del sistema.

4.1. Identificación de Riesgos

Se identificaron los siguientes riesgos principales:

- Cambios en los requerimientos del sistema.
- Problemas de comunicación con los usuarios.
- Subestimación del esfuerzo en funcionalidades complejas.
- Falta de experiencia técnica del equipo.
- Fallas técnicas en el entorno de desarrollo.
- Resistencia al cambio por parte del personal del hospital.
- Problemas de conexión o pérdida de datos.

4.2. Análisis de Riesgos

Cada riesgo fue evaluado considerando su probabilidad e impacto:

Riesgo	Probabilidad	Impacto
Cambios en los requerimientos	Alta	Alto
Subestimación del esfuerzo	Alta	Alto
Problemas de comunicación	Media	Alto
Problemas de datos o conexión	Media	Alto
Falta de experiencia técnica	Media	Moderado
Resistencia al cambio	Media	Moderado
Fallas técnicas graves	Baja	Alto

4.3. Estrategias de Mitigación

Para cada riesgo se definen acciones preventivas:

Riesgo	Estrategia de Mitigación
Cambios en requerimientos	Reuniones periódicas con usuarios clave (Product Owner) y validación continua de funcionalidades en cada *sprint review*. Definición de un proceso formal de gestión de cambios.
Problemas de comunicación	Uso de canales de comunicación definidos (ej. reuniones diarias, plataforma de seguimiento). Documentación clara de requerimientos (Historias de Usuario) y acuerdos.
Subestimación del esfuerzo	Planificación con márgenes de tiempo (buffers) para tareas críticas. Revisión y *grooming* del *Product Backlog* antes de cada *sprint*. Uso de técnicas de estimación por consenso (ej. Planning Poker).
Falta de experiencia técnica	Capacitación del equipo en tecnologías clave. Consulta y apoyo de recursos externos (expertos, foros). Fomento del desarrollo en pares (*pair programming*).
Fallas técnicas	Uso de entornos de desarrollo y prueba separados y estables. Implementación de control de versiones (Git) y copias de seguridad periódicas del código.
Resistencia al cambio	Capacitación anticipada y continua a usuarios finales (médicos, administrativos). Diseño de interfaces simples e intuitivas centradas en la usabilidad. Involucrar a líderes de opinión del hospital en el *feedback*.
Problemas de datos	Implementación de *backups* automáticos diarios de la base de datos. Definición de políticas de control de acceso y privilegios de usuario rigurosas. Monitoreo constante de la conexión y rendimiento del servidor.

4.4. Monitoreo de Riesgos

Los riesgos serán revisados en cada *sprint* mediante:

- Seguimiento del avance del proyecto y métricas de desempeño.
- *Feedback* continuo del equipo de desarrollo y los usuarios clave (durante la *Daily Scrum* y *Sprint Review*).
- Identificación de nuevos riesgos o cambios en la probabilidad/impacto de los riesgos existentes.

Esto permitirá ajustar las estrategias de mitigación y reducir el impacto de posibles problemas.

---

5. Evaluación de Arquitecturas, Selección y Justificación

Para el desarrollo del sistema Clinicks, se consideró la evaluación de diversas arquitecturas de *software* con el objetivo de seleccionar la que mejor se adapte a los requerimientos de seguridad, escalabilidad y mantenibilidad, aspectos cruciales en un entorno de gestión de datos de salud.

5.1. Arquitecturas Seleccionadas y Justificación

Se ha optado por un enfoque combinado, implementando la **Arquitectura por Capas** dentro de un esquema **Cliente-Servidor**.

5.1.1. Arquitectura por Capas

Justificación:

La principal razón para la adopción de la arquitectura por capas es la necesidad de garantizar la **seguridad, la integridad y la trazabilidad** en el manejo de la información sensible de los pacientes. Este modelo fuerza una separación estricta de las responsabilidades, lo que resulta fundamental:

- **Seguridad y Aislamiento de Datos:** La capa de Acceso a Datos (DAL) se aísla de la capa de Presentación, asegurando que la única forma de interactuar con la base de datos sea a través de la Lógica de Negocio. Esto minimiza el riesgo de inyecciones SQL u otros ataques dirigidos a los datos de salud (Historias Clínicas, DNI, diagnósticos, etc.).
- **Mantenibilidad y Escalabilidad:** Permite modificar o actualizar una capa sin afectar significativamente a las demás. Por ejemplo, si se necesita cambiar la base de datos (capa de datos), la Lógica de Negocio permanece intacta, lo que facilita el mantenimiento a largo plazo.
- **Reutilización:** La Lógica de Negocio puede ser reutilizada por diferentes interfaces de usuario (web, móvil) o incluso por otros sistemas en el futuro.

Capas Clave:

1. **Capa de Presentación (UI):** Responsable de la interacción con el usuario (formularios, visualización de historias clínicas).
2. **Capa de Lógica de Negocio (BLL):** Contiene las reglas del negocio (validación de datos de pacientes, lógica de asignación de habitaciones, reglas de registro de eventos clínicos).
3. **Capa de Acceso a Datos (DAL):** Gestiona la conexión y las operaciones CRUD (Crear, Leer, Actualizar, Borrar) con la base de datos.

5.1.2. Arquitectura Cliente-Servidor

Justificación:

El modelo Cliente-Servidor es el más adecuado para el contexto hospitalario, donde múltiples usuarios (médicos, enfermeros, administrativos) acceden al sistema de manera concurrente desde diferentes dispositivos:

- **Centralización de Datos y Seguridad:** El servidor actúa como punto central de control para los datos. Esto es vital para asegurar que todos los usuarios accedan a la misma versión de la información del paciente en tiempo real y que las políticas de seguridad (autenticación, autorización) se apliquen de manera uniforme y rigurosa en un único lugar.
- **Eficiencia de Recursos:** La mayor parte del procesamiento pesado (lógica de negocio y acceso a la base de datos) se ejecuta en el servidor, permitiendo que las estaciones de trabajo cliente sean ligeras y menos potentes.
- **Gestión de Acceso:** Facilita la implementación de la gestión de roles y privilegios, asegurando que solo el personal autorizado pueda registrar, consultar o modificar información crítica del historial clínico.

Esta combinación arquitectónica (Arquitectura por Capas dentro de un esquema Cliente-Servidor) proporciona la robustez, seguridad y estructura modular necesarias para gestionar eficientemente los datos sensibles del entorno de salud.